

在算法竞赛（如 ACM/ICPC, NOIP, LeetCode）中，熟练掌握 C++ 的自定义比较器是非常重要的。根据不同的使用场景，竞赛中最常用的比较器模板主要有三种：**结构体内重载运算符**、**Lambda 表达式**和**仿函数 (Functor)**。

以下是为你整理的实用模板和避坑指南：

1. 结构体内重载 < 运算符 (最常用、最推荐)

适用于自定义数据类型（如坐标、线段、图的边等）。只要重载了 <，就可以直接使用 `std::sort`、`std::set` 和 `std::map`，无需额外传参。

C++

```
1 struct Node {
2     int id;
3     int score;
4     int time;
5
6     // 必须加 const，且参数也必须是 const 引用
7     bool operator<(const Node& other) const {
8         if (score != other.score) {
9             return score > other.score; // 分数高的排前面（降序）
10        }
11        return time < other.time;      // 分数相同时，用时少的排前面（升序）
12    }
13 };
14
15 // 使用示例：
16 // vector<Node> v;
17 // sort(v.begin(), v.end());
18 // set<Node> s;
```

2. Lambda 表达式 (最快捷)

适用于不想单独写结构体，或者对 `vector<int>`、`vector<pair<int, int>>` 等标准库类型进行局部临时排序。

C++

```
1 vector<pair<int, int>> v;
2
3 // 使用 Lambda 表达式进行自定义排序
4 sort(v.begin(), v.end(), [](const pair<int, int>& a, const pair<int, int>& b) {
5     if (a.first != b.first) {
6         return a.first < b.first; // 第一关键字升序
7     }
8     return a.second > b.second;   // 第二关键字降序
9 });
```

技巧：在 Lambda 参数中加上 `const Type&` 可以避免大对象的拷贝，提升常数时间。

3. 仿函数 Functor (专用于 `priority_queue`)

`std::priority_queue` 无法直接传入 Lambda 表达式（在 C++20 之前非常麻烦），因此通常使用仿函数。

竞赛最大天坑警告： `priority_queue` 默认是大根堆（顶部元素最大）。如果你想把它变成小根堆，**比较器的逻辑需要反着写**（或者理解为返回 `true` 时，`a` 的优先级比 `b` 低，会被放在更下面）。

C++

```
1 struct Node {
2     int u, dist;
3 };
4
5 // 自定义比较器
6 struct cmp {
7     bool operator()(const Node& a, const Node& b) const {
8         // 注意这里是 >
9         // 含义: 如果 a.dist > b.dist, 则 a 的优先级低于 b, a 会被放到堆的下层
10        // 从而实现堆顶是 dist 最小的元素 (小根堆/Dijkstra常用)
11        return a.dist > b.dist;
12    }
13 };
14
15 // 使用示例:
16 // 优先队列: 数据类型, 底层容器, 比较器
17 priority_queue<Node, vector<Node>, cmp> pq;
```

⚠ 竞赛必读：比较器的严格弱序 (Strict Weak Ordering)

无论使用哪种方法，比较函数**必须**满足“严格弱序”，否则会导致 `std::sort` 运行时段错误 (Segmentation Fault) 或死循环 (RE)：

1. **永远不要使用 `<=` 或 `>=`**：当 `a` 和 `b` 相等时，比较函数必须返回 `false`。
2. **正确示例**：`return a.val < b.val;`
3. **错误示例**：`return a.val <= b.val;` (这在某些编译器或评测机环境下会导致直接崩溃)。